

C++中string类的常用方法实例总结 <https://www.jb51.net/article/240999.htm>

更新时间：2022年03月16日 10:45:00 作者：Goallegoal

string类是C++提供的抽象数据类型,其支持可变长字符串,下面这篇文章主要给大家总结介绍了关于C++中string类的常用方法,文中通过实例代码介绍的非常详细,需要的朋友可以参考下

目录 -

引言

C++ string类常用方法

总结

引言

C++中提供了一种新的数据类型——字符串类型(string)。实际上string并不是C++的基本类型,它是在C++标准库中声明的一个字符串类,用这种数据类型可以定义对象,每一个字符串变量都是string类的一个对象。标准库类型string表示可变长的字符序列,使用string类型必须首先包含它的头文件。

作为标准库的一部分, string定义在命名空间std中。

【例】

```
1 | #include<string> //注意这里没有.h  
2 | using namespace std;
```

string类的意义有两个:第一个是为了处理char类型的数组,并封装了标准C中的一些字符串处理的函数。而当string类进入了C++标准后,它的第二个意义就是一个容器。

C++ string类常用方法

1、string(const char *s) : 将 string 对象初始化为 s 指向的字符串

```
1 | string str("hello");
```

2、string(size_type n,char c) : 创建一个包含 n 个元素的 string 对象,其中每个元素都被初始化为字符 c

```
1 | string str(10, 'a');
```

3、string(const string &str)：将一个 string 对象初始化为 string 对象 str（复制构造函数）

```
1 | string str("abcde");  
2 | string str2(str);
```

4、string()：创建一个默认的 string 对象，长度为 0（默认构造函数）

```
1 | string str;
```

5、使用C语言风格字符串处理string对象

```
1 | string str = "hello!";
```

6、获取string对象的长度，C语言中使用strlen()来获取字符串长度，C++中使用str.size()或str.length()。

```
1 | string str("hello!");  
2 | int len1 = str.size();  
3 | int len2 = str.length();
```

7、将一个 string 对象赋值给另一个 string 对象

```
1 | string str("hello!");  
2 | string str2;  
3 | str2 = str;
```

8、string 对象的拼接

C 语言中使用 strcat、strncat 函数来进行字符串拼接操作，C++中可以采用以下方式：

```
1 | string str1("hello");  
2 | string str2("world");  
3 | string str3 = str1 + str2;
```

9、使用 += 来在一个 string 对象后面附加一个 string 对象、字符以及 C 风格的字符串

```
1 | string str("hello");  
2 | string str2("world");  
3 | str += str2;  
4 | str += 'a';  
5 | str += "abcd";
```

10、string.append() 函数，在string对象后添加一个string对象或c风格字符串。

```

1 | string str("hello");
2 | string str2("world");
3 | str.append(str2);
4 | str.append("abcd");

```

11、string.push_back() 函数来在一个 string 对象后面附加一个字符

```

1 | string str("hello");
2 | char ch = 'a';
3 | str.push_back(ch);

```

12、对于string对象的比较，可以直接使用关系运算符。

```

1 | string str1("abcd");
2 | string str2("abcd");
3 | if(str1 == str2)
4 |     break;

```

13、string对象的比较也可以使用string.compare() 方法

```

1 | int compare(const string&str) const;
2 | int compare(size_t pos, size_t len, const string&str) const;
3 | int compare(size_t pos, size_t len, const string&str, size_t subpos,
4 | int compare(const char * s) const;
5 | int compare(size_t pos, size_t len, const char * s) const;
6 | int compare(size_t pos, size_t len, const char * s, size_t n) const;
7 |
8 | // example
9 | string str1("hello world");
10 | string str2("hello boy");
11 | str1.compare(6, 3, str2, 6, 3);

```

14、使用 string.substr() 函数来获取子串

```

1 | string str("hello");
2 | string str2 = str.substr(3,2)

```

15、访问 string 字符串的元素

```

1 | string str("hello");
2 | cout << str[2] << endl;
3 | cout << str.at(2) << endl;

```

16、使用 string.find() 方法查找字符

```

1 | // 从字符串的 pos 位置开始（若不指定 pos 的值，则默认从索引 0 处开始），查
2 | // 如果找到，则返回该子字符串首次出现时其首字符的索引；否则，返回 string::
3 | // string 类将 npos 定义为保证大于任何有效下标的值。
4 | size_type find (const string& str, size_type pos = 0) const;
5 | size_type find (const char *s, size_type pos = 0) const;

```

```

6 | size_type find (const char *s, size_type pos, size_type n);
7 | size_type find (char ch, size_type pos = 0) const;

```

17、string.rfind() 与 string.find() 方法类似，只是查找顺序不一样，string.rfind() 是从指定位置 pos（默认为字符串末尾）开始向前查找，直到字符串的首部，并返回第一次查找到匹配项时匹配项首字符的索引。换句话说，就是查找子字符串或字符最后一次出现的位置。

18、string.find_first_of() 方法在字符串中从指定位置开始向后（默认为索引 0 处）查找参数中任何一个字符首次出现的位置

```

1 | string str("hello world");
2 | int pos = str.find_first_of("abcde");
3 | int pos = str.find_first_of("abcde", 1); // 第二个参数为位置

```

19、string.find_last_of() 方法在字符串中查找参数中任何一个字符最后一次出现的位置

20、string.find_first_not_of() 方法在字符串中查找第一个不包含在参数中的字符

21、string.find_last_not_of() 方法在字符串中查找最后一个不包含在参数中的字符

22、使用 string.insert() 进行插入操作

```

1 | string& insert (size_t pos, const string&str) ;
2 | // 在位置 pos 处插入字符串 str
3 |
4 | string& insert (size_t pos, const string&str, size_t subpos, size_t :
5 | // 在位置 pos 处插入字符串 str 的从位置 subpos 处开始的 sublen 个字符
6 |
7 | string& insert (size_t pos, const char * s) ;
8 | // 在位置 pos 处插入字符串 s
9 |
10 | string& insert (size_t pos, const char * s, size_t n) ;
11 | // 在位置 pos 处插入字符串 s 的前 n 个字符
12 |
13 | string& insert (size_t pos, size_t n, char c) ;
14 | // 在位置 pos 处插入 n 个字符 c
15 |
16 | iterator insert (const_iterator p, size_t n, char c);
17 | // 在 p 处插入 n 个字符 c，并返回插入后迭代器的位置
18 |
19 | iterator insert (const_iterator p, char c);
20 | // 在 p 处插入字符 c，并返回插入后迭代器的位置

```

23、使用 string.erase() 进行元素删除操作

```

1 | string& erase (size_t pos = 0, size_t len = npos); // 删除从 pos 处
2 | iterator erase (const_iterator p); // 删除 p 处的
3 | iterator erase (const_iterator first, const_iterator last); // 删除从

```

24、使用 getline() 函数来获取 string 输入

```
1 | string str;  
2 | getline(cin, str);
```

25、使用 `string.empty()` 函数判断字符串是否为空

26、使用 `string.swap()` 函数交换两个字符串

```
1 | string str1 = "hello";  
2 | string str2 = "HELLO";  
3 | str1.swap(str2);
```

27、`string.back()` 获取或修改字符串最后一个字符

```
1 | string str("abcd");  
2 | char b = str.back();  
3 | str.back() = 'e';
```

28、`string.front()` 获取或修改字符串第一个字符

29、`string.pop_back()` 删除字符串最后一个元素